

5교시: 실패 분석 라이프사이클 - reproduce, observe, hypothesize, fix, recheck, prevent

수업 목표

- 실패 분석을 6단계로 기록한다.
- symptom과 root cause candidate를 구분한다.
- 없는 파일 요청으로 404를 만들고 RCA를 작성한다.

오늘 반드시 가져갈 것

필수 개념	왜 필수인가	놓치면 생기는 문제	확인 기록
재현	같은 실패를 다시 만들 수 있어야 비교가 가능하다.	고쳤는지 확인할 기준이 없다.	실패 URL과 명령
증상과 원인 후보	보이는 현상과 가능한 원인을 분리한다.	404, 서버 종료, 포트 충돌을 섞어 말한다.	상태 코드, 로그, 원인 후보
재확인	수정 후 같은 방식으로 다시 확인한다.	"고쳤다"는 말만 남고 검증이 없다.	fixed check command/상태 코드
예방	같은 실패를 줄이는 문서나 체크를 남긴다.	같은 실수가 반복된다.	README 수정, 주의 문장

챌린저 복구 기준

- 404는 서버가 죽었다는 뜻이 아니다. 요청은 받았지만 해당 경로의 파일을 찾지 못한 것이다.
- connection refused는 먼저 서버 프로세스와 포트를 본다.
- RCA는 큰 장애 보고서가 아니라 작은 실패를 다시 줄이는 연습 기록이다.

50분 흐름

Time	Activity
0-5분	README 재현성 checklist 확인
5-15분	RCA와 symptom/cause 차이 설명
15-30분	404 실패 재현과 log 관찰
30-40분	fix/recheck/prevent 기록
40-50분	관찰 가능성 signal로 연결

0-5분 README 재현성 checklist 확인

- 진행: README 재현성 checklist 확인
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

상세 설명

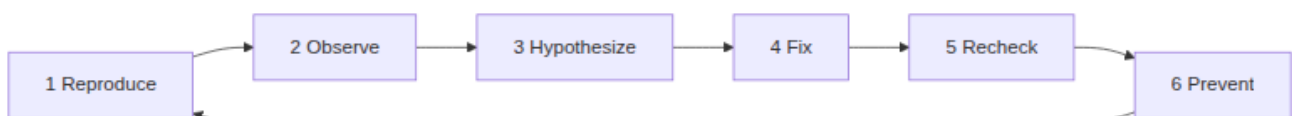
RCA는 누군가를 탓하는 문서가 아니라 같은 실패를 줄이기 위한 학습 기록이다. 첫 단계는 실패를 재현하는 것이다. 재현할 수 없으면 관찰도 비교도 어렵다. 다음으로 log, 상태 코드, command output을 관찰하고, 원인 후보를 세운다. fix를 적용한 뒤 같은 check로 다시 확인하고, 마지막에는 재발 방지 방법을 남긴다.

오늘은 없는 파일을 요청해 404를 만든다. 404는 서버가 죽었다는 뜻이 아니라 server가 요청을 받았지만 해당 경로의 resource를 찾지 못했다는 뜻이다. 이 차이를 이해하면 connection refused, 404, 500을 섞어 말하지 않게 된다.

Visual 1: RCA lifecycle



이 이미지는 RCA를 책임 추궁이 아니라 관찰 가능한 증거로 문제를 좁히는 절차로 다룬다. 특히 Timeline과 Recheck를 분리해 “수정했다”와 “수정이 확인됐다”를 구분한다.



RCA는 한 번 쓰고 끝나는 보고서가 아니라 같은 실패를 줄이는 반복 구조다. 오늘은 작은 404로 이 흐름을 연습한다.

5-15분 RCA와 symptom/cause 차이 설명

- 진행: RCA와 symptom/cause 차이 설명
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

Visual 2: 상태 코드별 첫 판단

Symptom	프로세스 상태 추정	먼저 볼 확인 기록
connection refused	서버 프로세스가 없거나 포트가 다름	서버 터미널, 포트
404	프로세스는 응답함, 경로/resource 문제 가능	URL 경로, ls 결과, request log
500	프로세스는 응답함, 내부 처리 실패 가능	error log, recent change
브라우저 blank	HTML 내용 또는 wrong file 가능	curl, cat index.html

상태 코드는 정답이 아니라 출발점이다. "어디가 고장났을 가능성이 높은가"를 좁히는 확인 기록으로 사용한다.

15-30분 404 실패 재현과 log 관찰

- 진행: 404 실패 재현과 log 관찰
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

RCA 템플릿

Step	Record
Reproduce	
Observe	
Hypothesize	
Fix	
Recheck	
Prevent	

명령 절차

서버가 실행 중인 상태에서 새 terminal을 연다.

```
curl -I http://localhost:8000/no-such-file.html
```

수정 방법은 새 기능 구현이 아니라 올바른 existing 경로로 확인하는 것이다.

```
curl -I http://localhost:8000/index.html
```

의도적 오류 로그 수집 절차

이번 시간에는 실수로 난 오류를 기다리지 않고, 일부러 작고 안전한 오류를 만든다. 없는 파일 요청은 시스템을 망가뜨리지 않으면서도 상태 코드와 서버 로그를 남기므로 RCA 연습에 적합하다.

1. 서버 터미널이 켜져 있는지 확인한다. 2. 새 터미널에서 실패 요청을 보낸다. 3. 서버 터미널에서 같은 시간대의 request log를 찾는다. 4. 실패 상태 코드와 요청 경로만 발췌한다. 5. 정상 URL로 다시 확인한다.

예상되는 실패 확인 기록은 다음과 같다. 실제 로그 형식은 Python 버전과 OS에 따라 조금 다를 수 있으므로, 문장 그대로 맞추는 것이 아니라 요청 경로와 상태 코드를 찾는다.

```
request: GET /no-such-file.html
status: 404
interpretation: server process is alive, requested file path does not exist
```

서버 로그에 GET /no-such-file.html이 보이고 curl -I 결과가 404라면, 이 실패는 프로세스 종료 문제가 아니라 경로/resource 문제로 분류한다. 이 분류가 중요하다. 원인 후보가 다르다면 다음 확인 명령도 달라진다.

실패 증상	잘못된 결론	더 나은 첫 판단
404	서버가 죽었다.	서버는 응답했다. URL 경로와 파일 존재 여부를 본다.
connection refused	파일이 없다.	서버 프로세스 또는 포트가 먼저 의심된다.
수정 후 예전 본문	curl이 틀렸다.	저장한 파일, 실행 경로, cache를 본다.

수정 후 재확인 기록

RCA에서 fix는 큰 코드를 새로 만드는 일이 아니다. 이번 실습의 fix는 요청 경로를 존재하는 파일로 바꾸거나, README에 올바른 URL을 명시하는 것이다. 중요한 것은 fix 다음에 같은 관찰 기준으로 다시 확인하는 것이다.

```
curl -I http://localhost:8000/index.html
curl http://localhost:8000/index.html
```

재확인 기록에는 200 상태 코드와 예상 본문 일부가 함께 있어야 한다. 상태 코드만 200이어도 본문이 예전 내용이면 수정 확인은 끝난 것이 아니다.

30-40분 fix/recheck/prevent 기록

- 진행: fix/recheck/prevent 기록

- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

확인 질문

- 이번 실패의 symptom은 무엇인가?
- root cause candidate는 프로세스, 포트, file 경로 중 어디에 가까운가?
- recheck에 같은 명령을 다시 써야 하는 이유는 무엇인가?
- 오류 로그에서 전체 복사 대신 어떤 부분만 발췌해야 안전한가?

다음 주차 매핑

Docker에서는 wrong file 경로가 image build context 문제로 나타날 수 있고, Kubernetes에서는 wrong 경로가 readiness 실패로 보일 수 있다. AWS 배포에서는 ALB target health와 application 상태 코드를 분리해서 봐야 한다.

예상 결과

- 없는 파일은 보통 404 상태 코드를 반환한다.
- 서버 terminal에는 /no-such-file.html 요청 log가 남는다.
- /index.html 요청은 200 상태 코드를 반환해야 한다.

흔한 오해

오해	교정
404는 서버 프로세스가 죽은 것이다.	server가 응답했으므로 프로세스는 살아 있다. 경로/resource 문제다.
RCA는 큰 장애 때만 쓴다.	작은 실패에서 연습해야 큰 장애 때 구조적으로 기록할 수 있다.
fix만 하면 기록은 필요 없다.	prevent가 없으면 같은 실수가 반복된다.

40-50분 관찰 가능성 signal로 연결

- 진행: 관찰 가능성 signal로 연결
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

실습 확인 기록

확인 항목	값
failing URL	
failing 상태 코드	
server log excerpt	
fixed check URL	
fixed 상태 코드	
fixed body 확인	
prevention note	

학술 근거와 DevOps insight

SRE와 incident management에서는 증상, 영향, 원인, 조치, 재발 방지를 분리해 기록한다. 작은 404 분석도 같은 구조를 사용한다. 현업에서 좋은 RCA는 정답 맞추기가 아니라 확인 기록을 근거로 원인 후보를 좁히는 과정이다.

평가 기준

기준	2점 확인 기록
50분 참여	시간 흐름에 맞춰 설명, 활동, 산출물 작성에 참여했다.
증거 산출	수업에서 요구한 note, command, table, blocker 중 해당 산출물을 구체적으로 남겼다.
전이 연결	오늘 개념이 Week2~5 기술 또는 자기 산출물과 어떻게 연결되는지 한 문장 이상 설명했다.

공식/학술 근거 링크

- Google SRE Book: Postmortem Culture, <https://sre.google/sre-book/postmortem-culture/> - RCA를 blame이 아니라 학습 가능한 확인 기록으로 남기는 기준이다.
- RFC 9110: HTTP Semantics, <https://datatracker.ietf.org/doc/html/rfc9110> - 404와 같은 상태 코드를 원인 후보 분리에 사용하는 공식 기준이다.
- Google SRE Book: Introduction, <https://sre.google/sre-book/introduction/> - monitoring, emergency response, change management를 운영 책임으로 보는 근거다.